TU Wien
Institute of Logic and Computation
Algorithms and Complexity Group

Heuristic Optimization Techniques, WS 2026

Programming Exercise: Assignment 1

v1, 16-09-2026

Work on the problem specified in the separate "Problem Description and General Information" document within your programming exercise group. You are encouraged to split tasks about evenly, but everybody has to understand and to be able to explain all the concepts involved, your overall implementation, and the submitted report. Hand in your complete report, submit the solutions for at least three of the instances in the competition, and upload your source code via TUWEL by **Sunday, 29. 11. 2026, 23:55**. For questions please contact us via `heuopt@ac.tuwien.ac.at`.

The first programming assignment is to develop your *own* construction and local search based heuristics for the *Resource-Constrained k-Subtree Problem* (RC-k-ST). Recall that the RC-k-ST is a minimization problem in which you decide *both* which k nodes to span *and* which edges to use for connecting them, and that the resource constraints couple these two decisions globally: exchanging a single node can render the current edge set infeasible. The subtasks for this exercise are:

1. Develop a meaningful deterministic construction heuristic.
2. Derive a randomized construction heuristic to be applied iteratively, yielding diverse promising solutions.
3. Develop a Pilot or Beam Search based heuristic.
4. Develop or make use of a framework for basic local search which is able to deal with
   - different neighborhood structures
   - different step functions (first-improvement, best-improvement)
5. Develop at least three different meaningful neighborhood structures.
6. Develop or make use of a Variable Neighborhood Descent (VND) framework which uses your neighborhood structures.
7. Implement a Greedy Randomized Adaptive Search Procedure (GRASP) using your randomized construction heuristic and an effective neighborhood structure with one step function or (a variant of) your VND. Note that the union of existing neighborhood structures also constitutes a (composite) neighborhood structure.
8. Implement ***one*** of the following metaheuristics:
   - General Variable Neighborhood Search (GVNS) on top of your VND
   - Simulated Annealing (SA)
   - Tabu Search (TS)
   - Configuration Checking (CC)
9. Use delta-evaluation. Explain which steps in your algorithm use delta-evaluation and describe why delta-evaluation generally improves performance here. If possible analyze the asymptotic runtime of a step using delta-evaluation, and of a step without delta-evaluation.

10. Perform some experimental manual tuning of relevant algorithmic parameters to find sensible parameter settings for your final experiments. Relevant parameters may be related to the degree of randomization, neighborhood structure sizes, probabilities for the random step function in composite neighborhood structures, the cooling schedule, the tabu list length and its variation, etc. Report how you derived these values and the impact of different settings on the solution quality for a selected meaningful subset of instances. Use the instances in `instances/train/` for your tuning and report your final results on the separate `instances/test/`.

11. **Overview.** Run experiments and compare **all** the algorithms on different criteria on the instances of different sizes provided in TUWEL; consider a sufficient number of instances to derive meaningful conclusions. Show the result in a concise table:

    (a) Runtime

    (b) Objective value

    (c) Gap to the best objective value you obtained for that instance with *any* of your algorithms

    (d) Number of iterations and of evaluated neighbors within the given time budget

12. **Controlled comparisons.** Isolate single design decisions by keeping everything else fixed, so that an observed difference can be attributed to that one choice:

    (a) Deterministic vs. randomized construction heuristic vs. GRASP: How much longer does GRASP take, does it improve results, what degree of randomization is good for GRASP? How much worse/better is the randomized construction heuristic compared to the deterministic one?

    (b) Deterministic construction heuristic vs. Pilot/Beam Search: How large is the gain in solution quality relative to the additional runtime? What is the effect of the number β of nodes kept per level (Beam Search) or of the rollout length γ (Pilot)?

    (c) Use the solution of the deterministic construction heuristic as a starting point to test the other implementations:

        i. Local search for at least three selected (possibly composite) neighborhood structures using each of the two step functions (i.e., at least six different algorithm variants).

        ii. VND. How does the order of neighborhood structures affect the solution quality?

        iii. For your metaheuristic from task 8, vary its most influential parameter (e.g., shaking size, cooling schedule, tabu list length) over a few settings and report the effect on solution quality and runtime.

    (d) Report on measures such as running time, iterations, average final objective and objective over time. How do the objective values and the remaining slack for the tightest resource l evolve over the course of the algorithm? Run each algorithm on each instance multiple times to reduce statistical variance.

    (e) Prepare plots showing the above measures in a concise fashion.

13. Write a report containing the description of your algorithms, the experimental results and what you conclude; see the *general information* document for more details.

Some questions to aid your development are:

- How is a solution represented best?
- How do you generate different solutions? Which parts of your algorithm can be reasonably randomized and how can you control the degree of randomization? Is it possible to randomize other parts and are your random solutions sufficiently diverse?
- How tight are the resource constraints in practice? In how many dimensions is the constraint active in your final solutions, and does a single dimension dominate?

- Are all the instances created in the same way? Are there any instances that are particularly hard to solve? Why?
- What values did you choose for your hyperparameters such as tabu list length or randomization factor?
- How do you handle infeasible solutions? Can your neighborhood structures produce them, and what does your choice cost you in search space connectivity or in evaluation time?
- Is every solution reachable, i.e. can subsequent, possibly non-improving, moves in your neighborhood structures reach every solution in the search space?
- Local search: How many iterations does it take on average to reach a local optimum? What does this say about your neighborhood structures? Does this change when using different step functions?
- How does delta evaluation work for your neighborhood structures?
- What is the time complexity to fully search one neighborhood of your neighborhood structures?
- VND: Does the order of your neighborhood structures affect the solution quality?

We hope you enjoy these tasks and wish you success!